

RA06: BD Distribuida

ÍNDICE

1. [Preparativos](#)
2. [Usuarios](#)
 - 2.1. [Tablespaces](#)
 - 2.2. [Tabla y particiones](#)
 - 2.3. [Creación de usuarios](#)
3. [Archivos de configuración](#)
 - 3.1. [sqlnet.ora](#)
 - 3.2. [tnsnames.ora](#)
 - 3.3. [listener.ora](#)
4. [Conexión con usuarios remotos](#)
5. [Trigger](#)
 - 5.1. [INSERT](#)
 - 5.2. [DELETE](#)
 - 5.3. [Comprobación desde otros usuarios vecinos](#)
6. [BD con los usuarios](#)
 - 6.1. [Distribución de usuarios](#)

Preparativos

En primer lugar, para poder empezar con la práctica, tendremos que crear un usuario **QUE NO TENGA EL ROL SYSDBA** y con todos los permisos posibles para que pueda crear tablas y triggers:

```
/* ductucx */
CREATE USER ductucx IDENTIFIED BY "ductucx1234!"
DEFAULT TABLESPACE ts_ductucx
TEMPORARY TABLESPACE temp
ACCOUNT UNLOCK;

GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE, CREATE SYNONYM, CREATE DATABASE LINK, CREATE ANY TRIGGER, CONNECT TO ductucx;
GRANT RESOURCE TO ductucx;
REVOKE SYSDBA FROM ductucx;

/* PARA ELIMINAR !!! */
/* DROP USER ductucx; */

/* TABLESPACE */
CREATE TABLESPACE ts_ductucx
DATAFILE 'ts_ductucx.dbf'
SIZE 100 M
AUTOEXTEND ON NEXT 10 M;
```

Probaremos a hacer la conexión...

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone...	Detalles de Cone...
admin_dieta	admin_dieta@//lo...
admin_empresa	admin_empresa...
admin_hospital	admin_hospital@...
ductucx	ductucx@//localh...
Evelin-Fiyano	evelin@//192.16...
Evelin-Marruecos	rumania_evelin@...
Evelin-Rusia	evelin@//192.16...
EVELINDietagana...	sys@//localhost:...
prueba1	prueba1@//local...
prueba2	prueba2@//local...
prueba3	prueba3@//local...

Name

ductucx

Color

Tipo de Base de Datos

Oracle

Información de usuario

Usuario de Proxy

Tipo de autenticación

Por defecto

Usuario

ductucx

Rol

valor por defecto

Contraseña

.....

☒ Guardar Contraseña

Tipo de Conexión

Básico

Detalles

Avanzado

Nombre del Host

localhost

Puerto

1521

☒ SID

xe

☐ Nombre del Servicio

Estado: Correcto

Ayuda

Guardar

Borrar

Probar

Conectar

Cancelar

Usuarios

Ahora con nuestro usuario administrador que acabamos de crear, crearemos a nuestros respectivos usuarios de los vecinos correspondientes:

Tablespaces

```
CREATE TABLESPACE ts_hugo10
DATAFILE 'ts_hugo10.dbf'
SIZE 100 M
AUTOEXTEND ON NEXT 10 M;

CREATE TABLESPACE ts_martin10
DATAFILE 'ts_martin10.dbf'
SIZE 100 M
AUTOEXTEND ON NEXT 10 M;

CREATE TABLESPACE ts_pablo10
DATAFILE 'ts_pablo10.dbf'
SIZE 100 M
AUTOEXTEND ON NEXT 10 M;
```

Tabla y particiones

```
CREATE TABLE ciudadanos (  
    id          NUMBER(10)          NOT NULL,  
    nombre      VARCHAR2(20)        NOT NULL,  
    apellido1   VARCHAR2(20)        NOT NULL,  
    apellido2   VARCHAR2(20),  
    dni         VARCHAR2(9)         NOT NULL,  
    CONSTRAINT pk_ciudadanos PRIMARY KEY (id),  
    CONSTRAINT uk_ciudadanos_dni UNIQUE (dni)  
)  
PARTITION BY RANGE (id) (  
    PARTITION hugo  
        VALUES LESS THAN (100)  
        TABLESPACE ts_hugo10,  
  
    PARTITION pablo  
        VALUES LESS THAN (200)  
        TABLESPACE ts_pablo10,  
  
    PARTITION martin  
        VALUES LESS THAN (MAXVALUE)  
        TABLESPACE ts_martin10  
);
```

Creación de usuarios

Aquí nos encargaremos de crear los usuarios con sus respectivos "tablespaces".

```
/* hugo */
ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE;
CREATE USER hugo IDENTIFIED BY hugo
DEFAULT TABLESPACE ts_hugo10
TEMPORARY TABLESPACE temp
ACCOUNT UNLOCK;

GRANT CONNECT, CREATE SESSION, CREATE TABLE, RESOURCE TO hugo;
GRANT CREATE DATABASE LINK TO hugo;

REVOKE CREATE TABLE FROM hugo CONTAINER = ALL;

/* martin */
ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE;
CREATE USER martin IDENTIFIED BY martin1
DEFAULT TABLESPACE ts_martin10
TEMPORARY TABLESPACE temp
ACCOUNT UNLOCK;

GRANT CONNECT, CREATE SESSION, CREATE TABLE, RESOURCE TO martin;
GRANT CREATE DATABASE LINK TO martin;

REVOKE CREATE TABLE FROM martin;

/* pablo */
ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE;
CREATE USER pablo IDENTIFIED BY pablo
DEFAULT TABLESPACE ts_pablo10
TEMPORARY TABLESPACE temp
ACCOUNT UNLOCK;

GRANT CONNECT, CREATE SESSION, CREATE TABLE, RESOURCE TO pablo;
GRANT CREATE DATABASE LINK TO pablo;

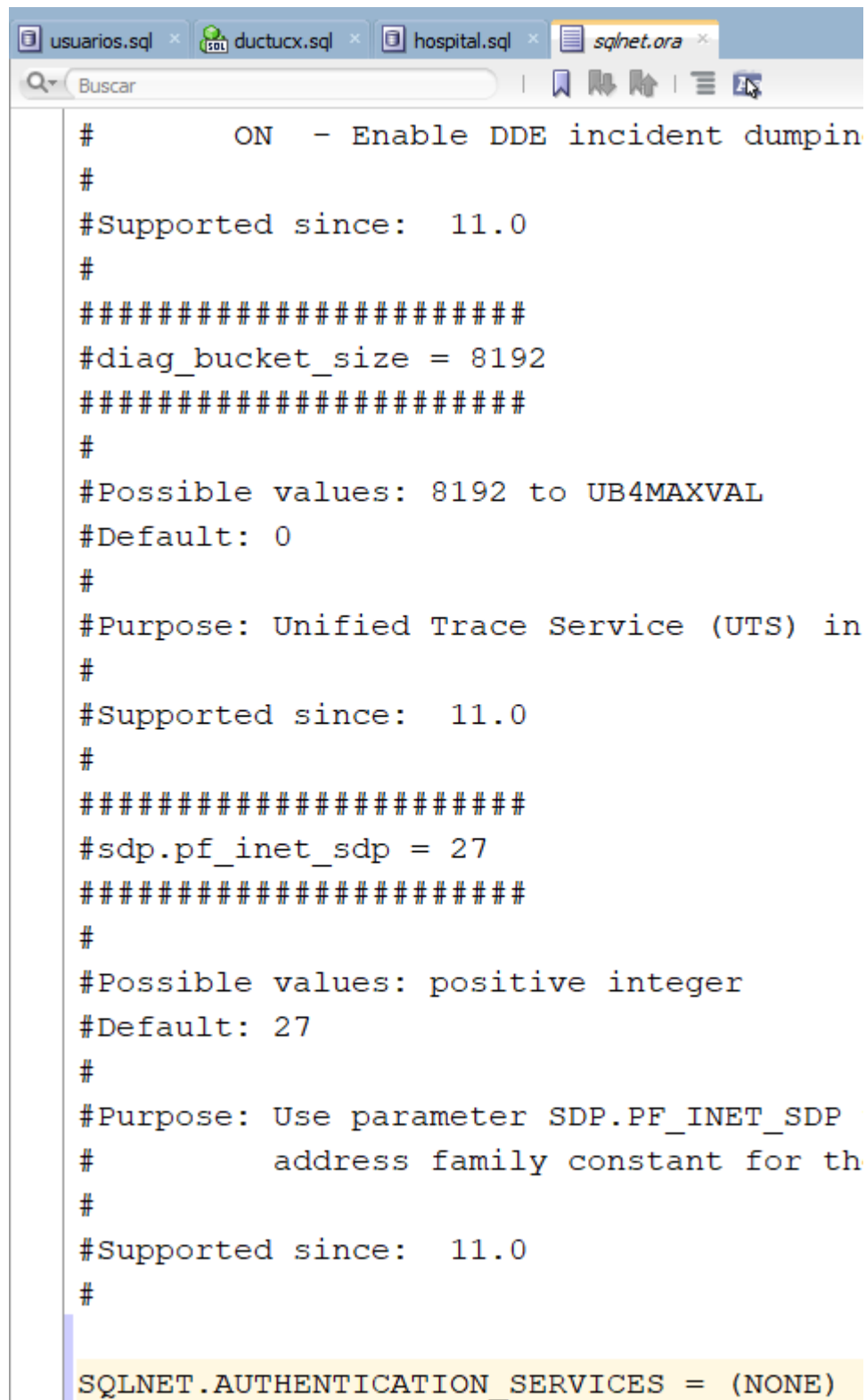
REVOKE CREATE TABLE FROM pablo CONTAINER = ALL;
```

Archivos de configuración

sqlnet.ora

En el archivo `sqlnet.ora` tendremos que añadir o modificar la siguiente línea:

`SQLNET.AUTHENTICATION_SERVICES = (NONE)`



```
#          ON  - Enable DDE incident dumpin
#
#Supported since:  11.0
#
#####
#diag_bucket_size = 8192
#####
#
#Possible values: 8192 to UB4MAXVAL
#Default: 0
#
#Purpose: Unified Trace Service (UTS) in
#
#Supported since:  11.0
#
#####
#sdp.pf_inet_sdp = 27
#####
#
#Possible values: positive integer
#Default: 27
#
#Purpose: Use parameter SDP.PF_INET_SDP
#          address family constant for th
#
#Supported since:  11.0
#
SQLNET.AUTHENTICATION_SERVICES = (NONE)
```

```
SQLNET.AUTHENTICATION_SERVICES = (NONE)
```

tnsnames.ora

Otro archivo que tendremos que cambiar es el `tnsnames.ora`, que tendremos que añadir las conexiones de nuestros vecinos para poder comunicarnos.

En el ejemplo tendremos que cambiar los nombres `ORCL_PROD` por uno que queramos asignar a nuestras conexiones, como por ejemplo `ORCL_<país>`.

```
ORCL_RUSIA =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.3.212) (PORT=1522))
    (CONNECT_DATA =
      (SERVER = DEDICATED
      (SERVICE_NAME = XE)
    )
  )

ORCL_FIYI =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.3.217) (PORT=1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED
      (SERVICE_NAME = XE)
    )
  )

ORCL_MARRUECOS =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.3.216) (PORT=1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED
      (SERVICE_NAME = XE)
    )
  )
```

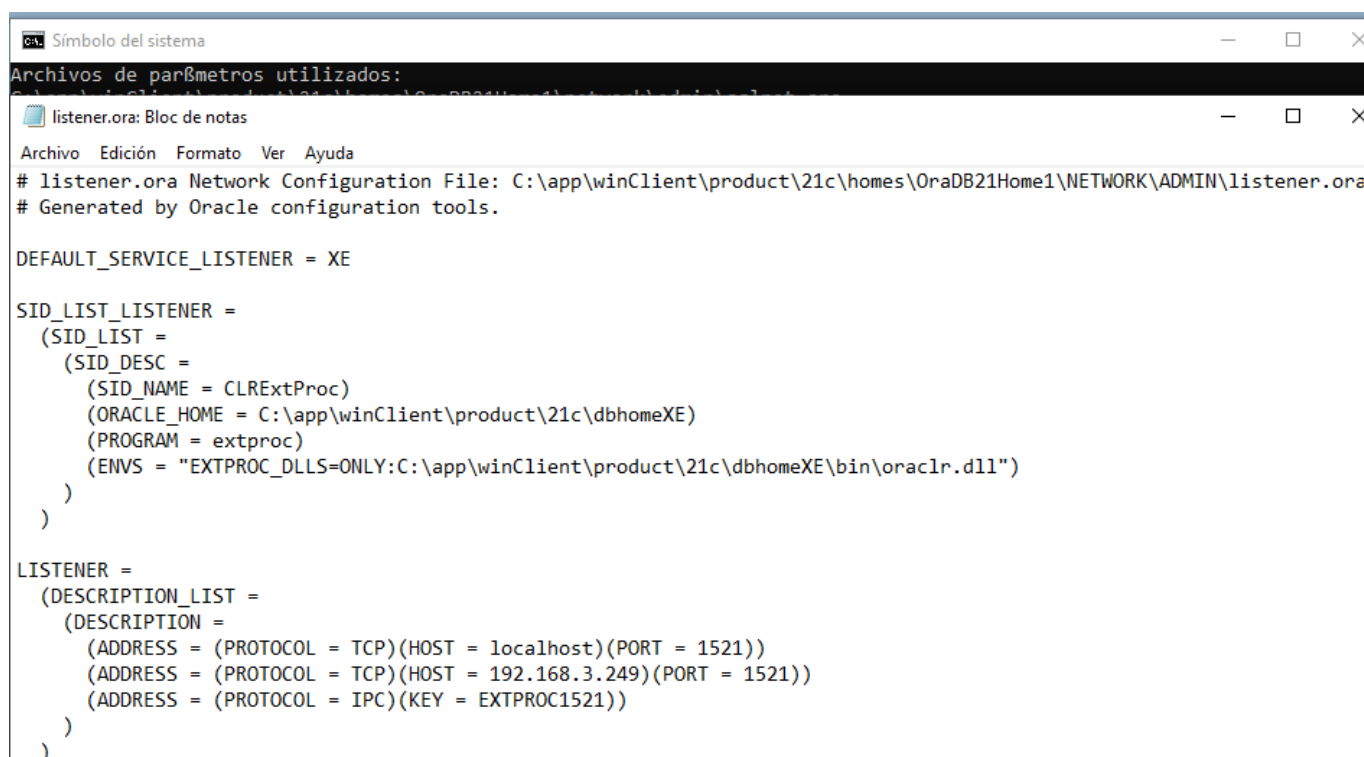
listener.ora

Y por último, nos aseguramos de que en **listener.ora** tengamos definido la IP y el puerto de nuestra máquina para que nuestros vecinos se puedan conectar.

Si nos fijamos bien encontraremos dos líneas muy importantes al final del fichero:

```
...  
    (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))  
    (ADDRESS = (PROTOCOL = TCP)(HOST = 192.168.3.249)(PORT = 1521))  
...
```

Tenemos que definir nuestro **localhost** y la **ip** en la que pueden otras máquinas encontrarnos.



```
Símbolo del sistema
Archivos de parámetros utilizados:
C:\app\winClient\product\21c\homes\OraDB21Home1\NETWORK\ADMIN\listener.ora

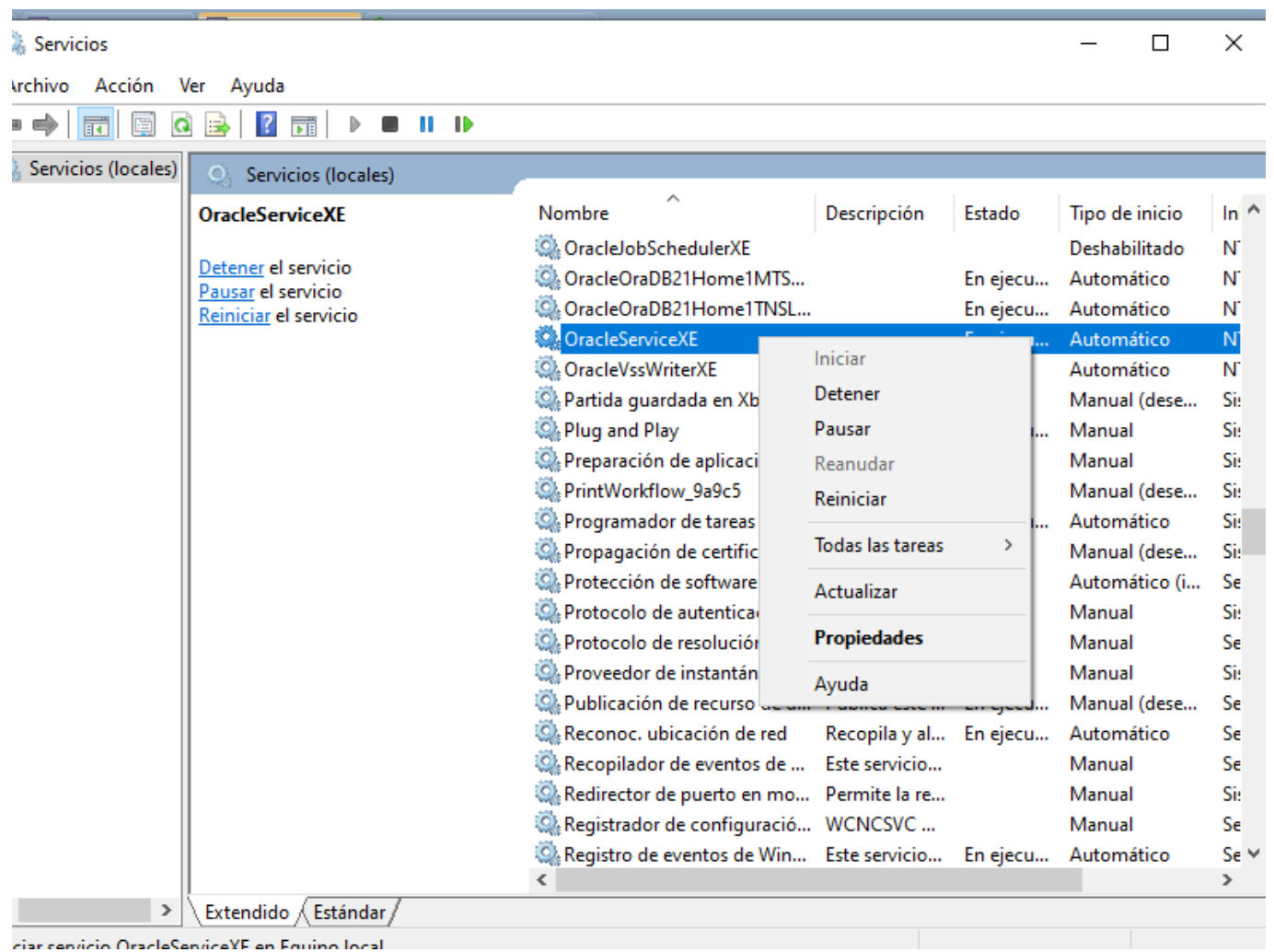
listener.ora: Bloc de notas
Archivo Edición Formato Ver Ayuda
# listener.ora Network Configuration File: C:\app\winClient\product\21c\homes\OraDB21Home1\NETWORK\ADMIN\listener.ora
# Generated by Oracle configuration tools.

DEFAULT_SERVICE_LISTENER = XE

SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = CLRExtProc)
      (ORACLE_HOME = C:\app\winClient\product\21c\dbhomeXE)
      (PROGRAM = extproc)
      (ENVS = "EXTPROC_DLLS=ONLY:C:\app\winClient\product\21c\dbhomeXE\bin\oraclr.dll")
    )
  )

LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))
      (ADDRESS = (PROTOCOL = TCP)(HOST = 192.168.3.249)(PORT = 1521))
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
    )
  )
)
```


Despues de hacer cualquier cambio en los ficheros de configuración, tendremos que reiniciar los servicios de Oracle para que aplique los nuevos cambios.



Y ya finalmente los demás equipos podrán conectarse al nuestro, y si ellos han seguido los mismos pasos, debería de poder comunicar:

```
Símbolo del sistema
TNS Ping Utility for 64-bit Windows: Version 21.0.0.0.0 - Production on 28-ENE-2026 13:02:56
Copyright (c) 1997, 2021, Oracle. All rights reserved.

Archivos de parámetros utilizados:
C:\app\winClient\product\21c\homes\OraDB21Home1\network\admin\sqlnet.ora

Adaptador TNSNAMES utilizado para resolver el alias
Intentando contactar con (DESCRIPTION = (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.3.212) (PORT=1522)) (CONNECT_DATA = (SERVER = DEDICATED (SERVICE_NAME = XE)))
Realizado correctamente (0 mseg)

C:\Users\winClient>tnsping ORCL_MARRUECOS

TNS Ping Utility for 64-bit Windows: Version 21.0.0.0.0 - Production on 28-ENE-2026 13:04:50
Copyright (c) 1997, 2021, Oracle. All rights reserved.

Archivos de parámetros utilizados:
C:\app\winClient\product\21c\homes\OraDB21Home1\network\admin\sqlnet.ora

Adaptador TNSNAMES utilizado para resolver el alias
Intentando contactar con (DESCRIPTION = (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.3.216) (PORT=1521)) (CONNECT_DATA = (SERVER = DEDICATED (SERVICE_NAME = XE)))
Realizado correctamente (7030 mseg)

C:\Users\winClient>tnsping ORCL_FIYI

TNS Ping Utility for 64-bit Windows: Version 21.0.0.0.0 - Production on 28-ENE-2026 13:05:09
Copyright (c) 1997, 2021, Oracle. All rights reserved.

Archivos de parámetros utilizados:
C:\app\winClient\product\21c\homes\OraDB21Home1\network\admin\sqlnet.ora

Adaptador TNSNAMES utilizado para resolver el alias
Intentando contactar con (DESCRIPTION = (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.3.217) (PORT=1521)) (CONNECT_DATA = (SERVER = DEDICATED (SERVICE_NAME = XE)))
Realizado correctamente (10 mseg)

C:\Users\winClient>tnsping ORCL_RUSIA

TNS Ping Utility for 64-bit Windows: Version 21.0.0.0.0 - Production on 28-ENE-2026 13:05:13
Copyright (c) 1997, 2021, Oracle. All rights reserved.

Archivos de parámetros utilizados:
```

Conexión con usuarios remotos

Con las credenciales que nos han otorgado nuestros vecinos, probamos a hacer la conexión:

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone...	Detalles de Cone...
admin_dieta	admin_dieta@//lo...
admin_empresa	admin_empresa...
admin_hospital	admin_hospital@...
ductucx	ductucx@//localh...
EVELINDietagana...	sys@//localhost:...
VecinoRuso	evelin@//192.16...

NameVecinoRusoColor

Tipo de Base de DatosOracle

Información de usuario

Usuario de Proxy

Tipo de autenticaciónPor defecto

UsuarioevelinRolvalor por defecto

ContraseñaevelinGuardar Contraseña

Tipo de ConexiónBásico

Detalles

Avanzado

Nombre del Host192.168.3.212

Puerto1522

SIDxe

Nombre del Servicio

Estado:

Ayuda

Guardar

Borrar

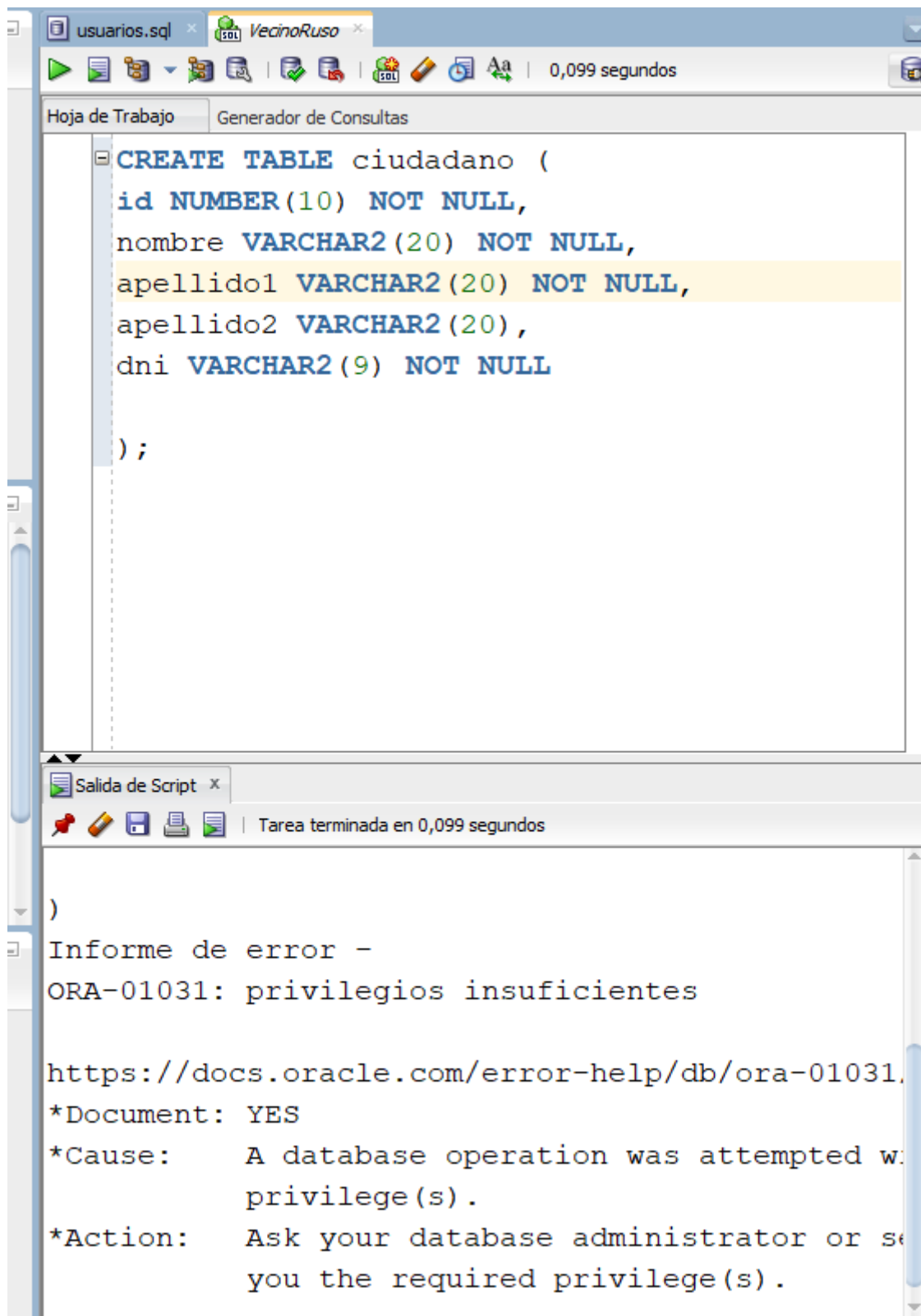
Probar

Conectar

Cancelar

/

Una vez que ingresemos dentro del usuario, podremos crear una tabla como ejemplo para verificar que se ha creado la partición correctamente. Una vez creada, el administrador de la base de datos nos tendrá que quitar los permisos para crear tablas:



The screenshot shows the SQL Developer interface. The top toolbar includes icons for running queries, saving, and other database operations. The main window is titled 'usuarios.sql' and 'VecinoRuso'. The 'Generador de Consultas' (Query Generator) tab is active, displaying the following SQL statement:

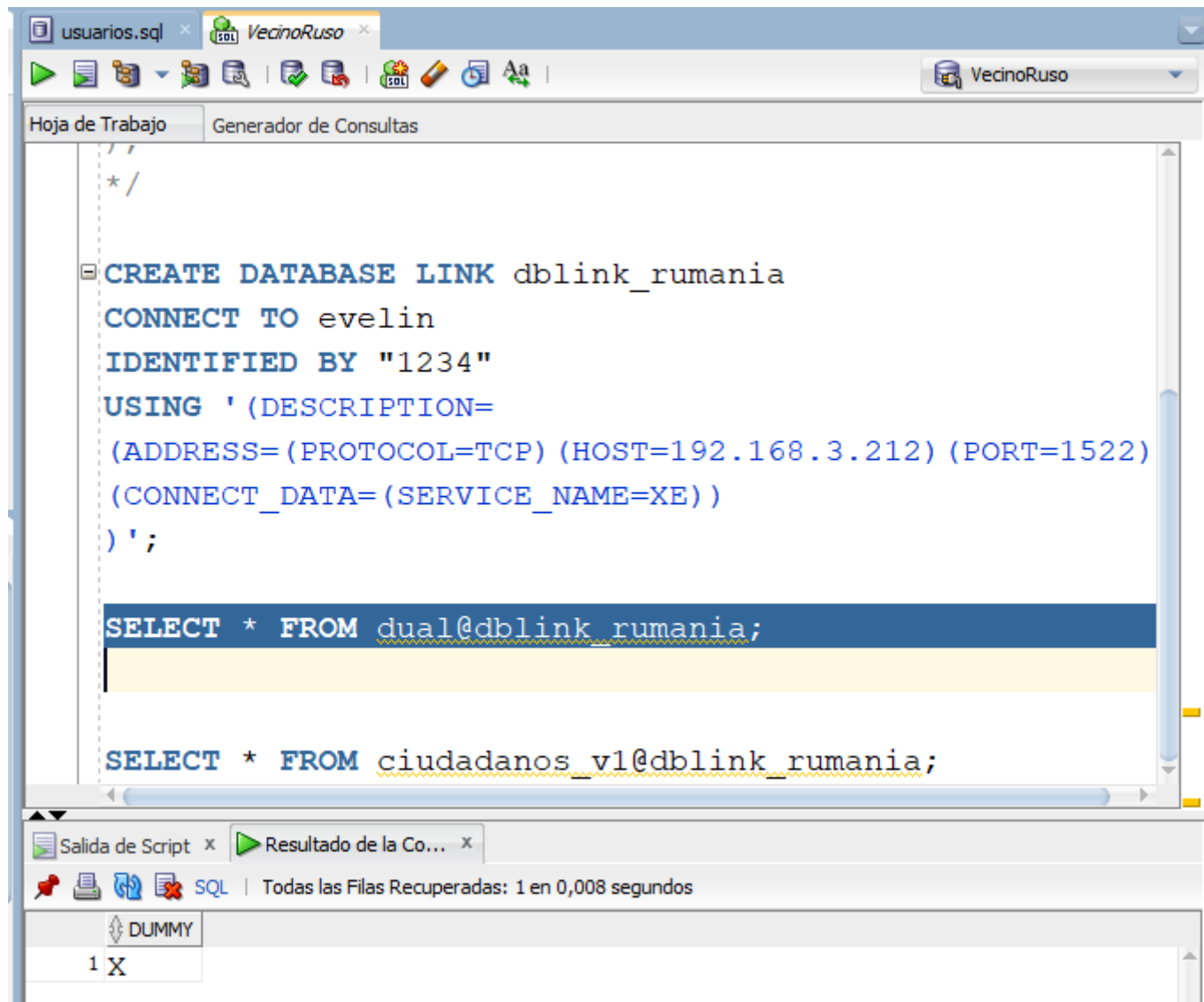
```
CREATE TABLE ciudadano (  
  id NUMBER(10) NOT NULL,  
  nombre VARCHAR2(20) NOT NULL,  
  apellido1 VARCHAR2(20) NOT NULL,  
  apellido2 VARCHAR2(20),  
  dni VARCHAR2(9) NOT NULL  
);
```

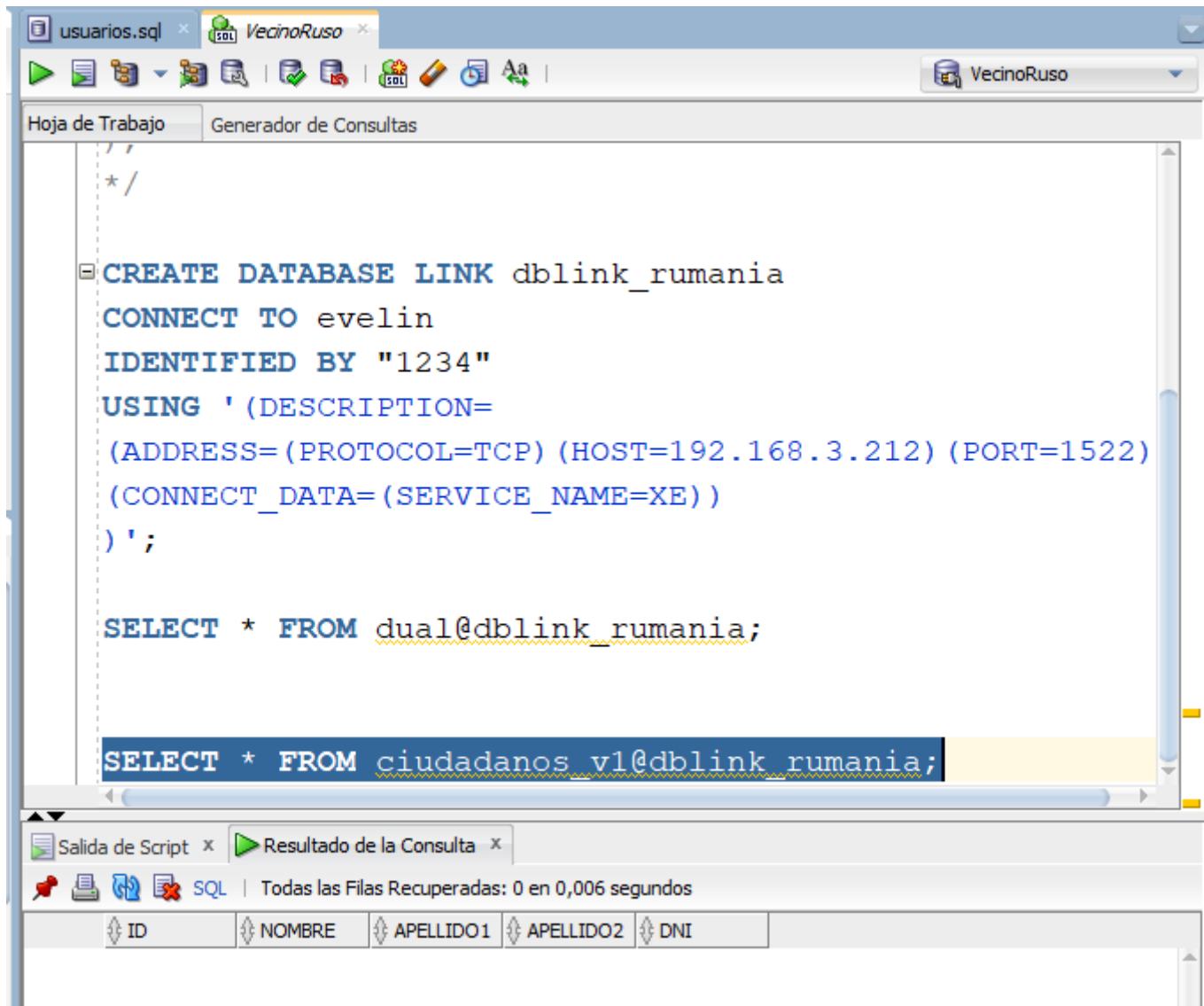
Below the query editor, the 'Salida de Script' (Script Output) window shows the execution result:

```
)  
Informe de error -  
ORA-01031: privilegios insuficientes  
  
https://docs.oracle.com/error-help/db/ora-01031  
*Document: YES  
*Cause: A database operation was attempted with insufficient  
privilege(s).  
*Action: Ask your database administrator or someone with  
DBA privileges to grant you the required privilege(s).
```

Para comprobar la tabla creada (en el caso de que tengamos una), crearemos un dblink

Un **DBLINK** (Database Link) básicamente sirve para que una base de datos Oracle pueda acceder a otra base de datos Oracle remota como si fuera local.





!!! ATENCIÓN !!! ==> En este ejemplo no muestra nada ya que la tabla ha sido eliminada y no encuentra datos, pero en la siguiente captura se muestra como mostrar los datos y comprobar que se ha creado en el tablespace correcto:

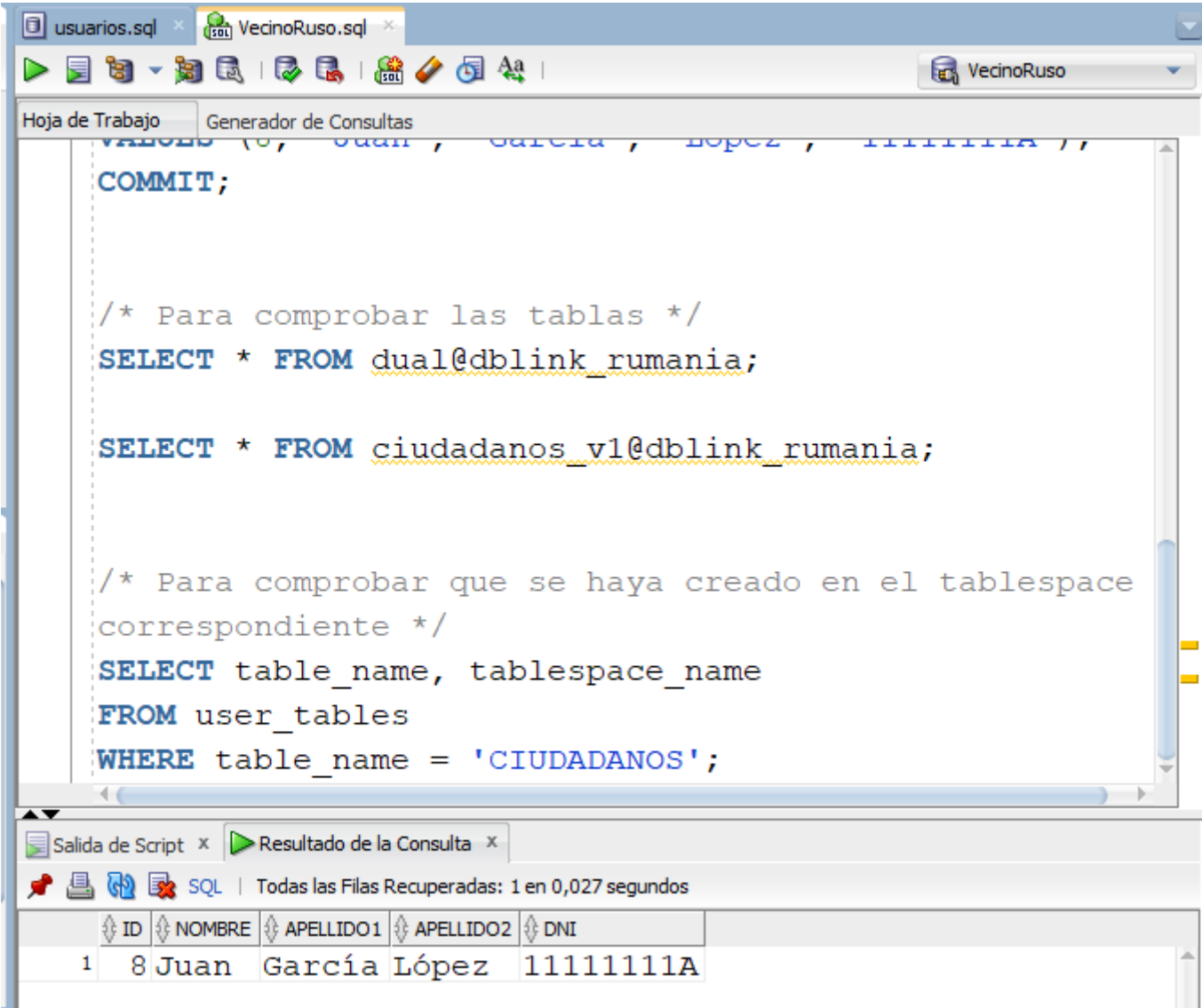
Pero para insertar los datos, dentro de nuestro usuario otorgado por el vecino, ejecutamos un **INSERT** y muy importante acabar con un **COMMIT**:

```

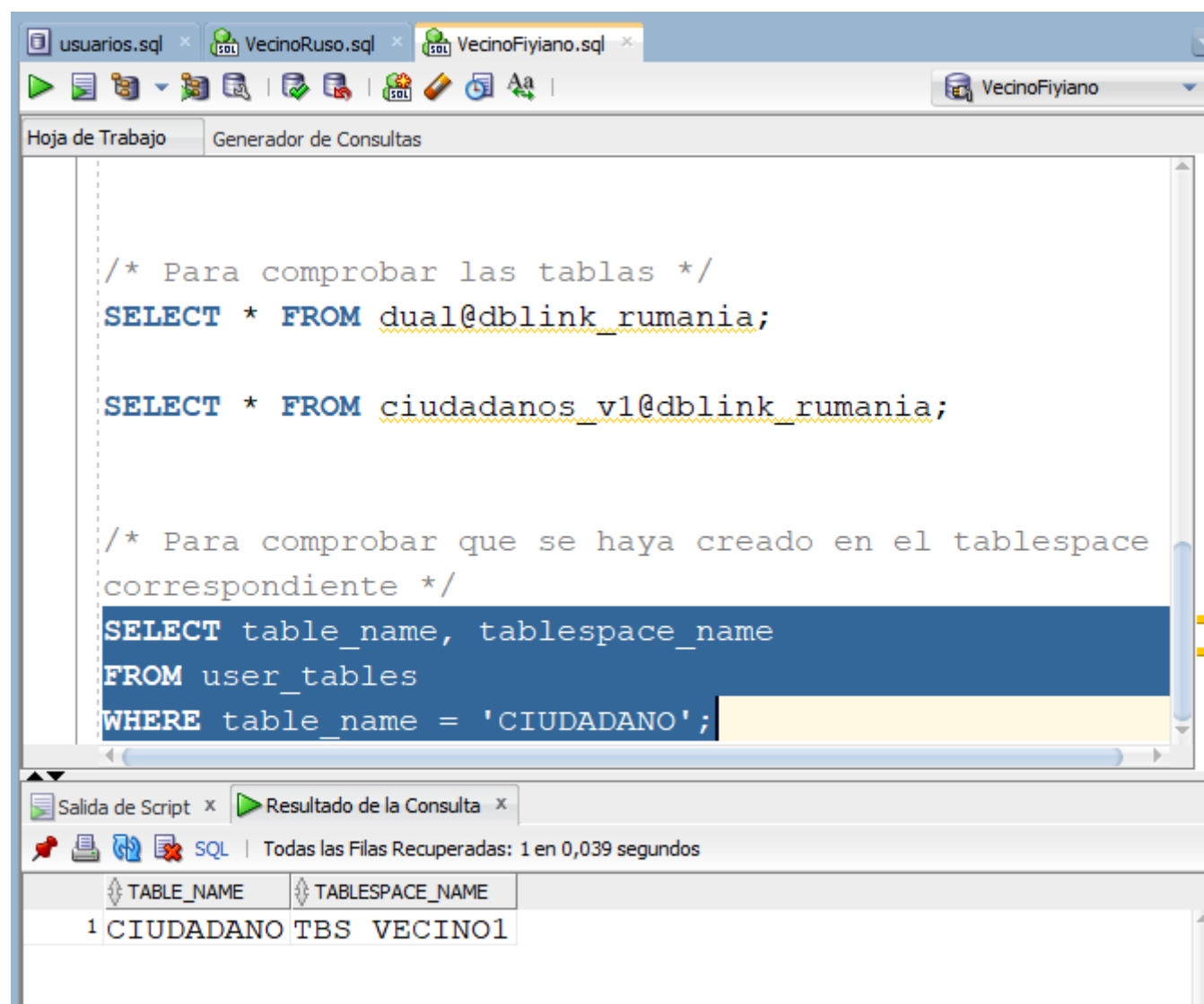
INSERT INTO <nombre_tabla>
VALUES (<id>, '<nombre>', '<apellido1>', '<apellido2>', '<dni>');
COMMIT;
  
```

```

INSERT INTO ciudadano
VALUES (8, 'Juan', 'García', 'López', '11111111A');
COMMIT;
  
```



De la siguiente manera podremos comprobar que se nos ha asignado el tablespace correcto:



The screenshot shows a SQL IDE window with three tabs: usuarios.sql, VecinoRuso.sql, and VecinoFiyiano.sql. The active tab is VecinoFiyiano.sql. The main editor area contains the following SQL script:

```
/* Para comprobar las tablas */  
SELECT * FROM dual@dblink_rumania;  
  
SELECT * FROM ciudadanos_v1@dblink_rumania;  
  
/* Para comprobar que se haya creado en el tablespace  
correspondiente */  
SELECT table_name, tablespace_name  
FROM user_tables  
WHERE table_name = 'CIUDADANO';
```

The last query is highlighted in blue. Below the editor, the 'Resultado de la Consulta' (Query Result) tab is active, showing the results of the last query. The results are displayed in a table with two columns: TABLE_NAME and TABLESPACE_NAME. The first row shows the table CIUDADANO is located in tablespace TBS_VECINO1.

TABLE_NAME	TABLESPACE_NAME
1 CIUDADANO	TBS_VECINO1

Y como en este caso nosotros disponemos de 3 vecinos en total, tendremos que probar a hacer la conexión y seguir exactamente los mismos pasos con los otros usuarios:

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone...	Detalles de Cone...
admin_dieta	admin_dieta@//lo...
admin_empresa	admin_empresa...
admin_hospital	admin_hospital@...
ductucx	ductucx@//localh...
EVELINDietagana...	sys@//localhost:...
VecinoFiyiano	evelin@//192.16...
VecinoRuso	evelin@//192.16...

NameVecinoFiyiano

Tipo de Base de DatosOrade

Información de usuario

Usuario de Proxy

Tipo de autenticaciónPor defecto

Usuarioevelin

Rolvalor por defecto

Contraseña*****

☐ Guardar Contraseña

Tipo de ConexiónBásico

Detalles

Avanzado

Nombre del Host192.168.3.217

Puerto1521

☒ SIDxe

☐ Nombre del Servicio

Estado: Correcto

Ayuda

Guardar

Borrar

Probar

Conectar

Cancelar

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone...	Detalles de Cone...
admin_dieta	admin_dieta@//lo...
admin_empresa	admin_empresa...
admin_hospital	admin_hospital@...
ductucx	ductucx@//localh...
EVELINDietagana...	sys@//localhost:...
prueba-hugo	hugo@//localhost...
VecinoFiyiano	evelin@//192.16...
VecinoMarroqui	rumania_evelin@...
VecinoRuso	evelin@//192.16...

NameVecinoMarroqui

Tipo de Base de DatosOrade

Información de usuario

Usuario de Proxy

Tipo de autenticaciónPor defecto

Usuariorumania_evelin

Rolvalor por defecto

Contraseña*****

☒ Guardar Contraseña

Tipo de ConexiónBásico

Detalles

Avanzado

Nombre del Host192.168.3.216

Puerto1521

☒ SIDxe

☐ Nombre del Servicio

Estado: Correcto

Ayuda

Guardar

Borrar

Probar

Conectar

Cancelar

/

Finalmente, gracias a los "dblinks" podremos consultar, crear o modificar las tablas sin ni si quiera entrar en el propio usuario otorgado por el vecino:

```
CREATE DATABASE LINK <nombre_link>
CONNECT TO <usuario>
IDENTIFIED BY <contraseña>
USING '<nombre_tnsname>';
```

```
CREATE DATABASE LINK link_hugo
CONNECT TO evelin
IDENTIFIED BY evelin
USING 'ORCL_FIYI';

SELECT * FROM dual@link_hugo;
SELECT * FROM ciudadano@link_hugo;

CREATE DATABASE LINK link_martin
CONNECT TO evelin
IDENTIFIED BY "1234"
USING 'ORCL_RUSIA';

SELECT * FROM dual@link_martin;
SELECT * FROM t_ciudadanos3@link_martin;

CREATE DATABASE LINK link_pablo
CONNECT TO rumania_evelin
IDENTIFIED BY rumania_evelin
USING 'ORCL_MARRUECOS';

SELECT * FROM dual@link_pablo;
SELECT * FROM ciudadanos@link_pablo;
```

```
/* PROBAR FUNCIONAMIENTO DEL DBLINK */  
SELECT * FROM dual@hugo_link;  
SELECT * FROM dual@pablo_link;  
SELECT * FROM dual@martin_link;
```

lida de Script x Resultado de la Consulta x

 SQL | Todas las Filas Recuperadas: 1 en 4,569 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1	8 Juan	García	López	11111111A

Trigger

Con el siguiente trigger lograremos copiar los INSERT, UPDATE y DELETE de nuestra tabla principal a las particionadas según el rango de su ID.

```
/* TRIGGER VERDADERO */  
CREATE OR REPLACE TRIGGER trg_ciudadanos  
AFTER INSERT OR UPDATE OR DELETE ON ciudadanos  
FOR EACH ROW  
BEGIN  
    -- =====  
    -- INSERT  
    -- =====  
    IF INSERTING THEN  
        IF :NEW.id < 100 THEN  
            INSERT INTO t_ciudadanos1@link_hugo  
            VALUES (:NEW.id, :NEW.nombre, :NEW.apellido1, :NEW.apellido2,  
:NEW.dni);  
  
            ELSIF :NEW.id < 200 THEN  
                INSERT INTO t_ciudadanos2@link_pablo  
                VALUES (:NEW.id, :NEW.nombre, :NEW.apellido1, :NEW.apellido2,  
:NEW.dni);  
  
            ELSE  
                INSERT INTO t_ciudadanos3@link_martin  
                VALUES (:NEW.id, :NEW.nombre, :NEW.apellido1, :NEW.apellido2,  
:NEW.dni);  
            END IF;  
  
    -- =====  
    -- DELETE  
    -- =====  
    ELSIF DELETING THEN  
        IF :OLD.id < 100 THEN  
            DELETE FROM t_ciudadanos1@link_hugo WHERE id = :OLD.id;  
  
            ELSIF :OLD.id < 200 THEN
```

```
        DELETE FROM t_ciudadanos2@link_pablo WHERE id = :OLD.id;

ELSE
    DELETE FROM t_ciudadanos3@link_martin WHERE id = :OLD.id;
END IF;

-- =====
-- UPDATE
-- =====
ELSIF UPDATING THEN
    -- Si cambia el ID, movemos el registro
    IF :OLD.id != :NEW.id THEN
        -- borrar del origen
        IF :OLD.id < 100 THEN
            DELETE FROM t_ciudadanos1@link_hugo WHERE id = :OLD.id;
        ELSIF :OLD.id < 200 THEN
            DELETE FROM t_ciudadanos2@link_pablo WHERE id = :OLD.id;
        ELSE
            DELETE FROM t_ciudadanos3@link_martin WHERE id = :OLD.id;
        END IF;

        -- insertar en destino
        IF :NEW.id < 100 THEN
            INSERT INTO t_ciudadanos1@link_hugo
            VALUES (:NEW.id, :NEW.nombre, :NEW.apellido1,
:NEW.apellido2, :NEW.dni);
        ELSIF :NEW.id < 200 THEN
            INSERT INTO t_ciudadanos2@link_pablo
            VALUES (:NEW.id, :NEW.nombre, :NEW.apellido1,
:NEW.apellido2, :NEW.dni);
        ELSE
            INSERT INTO t_ciudadanos3@link_martin
            VALUES (:NEW.id, :NEW.nombre, :NEW.apellido1,
:NEW.apellido2, :NEW.dni);
        END IF;

    ELSE

        ELSE
            -- mismo rango → solo UPDATE
            IF :NEW.id < 100 THEN
                UPDATE t_ciudadanos1@link_hugo
                SET nombre = :NEW.nombre,
                    apellido1 = :NEW.apellido1,
                    apellido2 = :NEW.apellido2,
                    dni = :NEW.dni
                WHERE id = :OLD.id;

            ELSIF :NEW.id < 200 THEN
                UPDATE t_ciudadanos2@link_pablo
                SET nombre = :NEW.nombre,
                    apellido1 = :NEW.apellido1,
                    apellido2 = :NEW.apellido2,
                    dni = :NEW.dni
                WHERE id = :OLD.id;
```

```
        ELSE
            UPDATE t_ciudadanos3@link_martin
            SET nombre = :NEW.nombre,
                apellido1 = :NEW.apellido1,
                apellido2 = :NEW.apellido2,
                dni = :NEW.dni
            WHERE id = :OLD.id;
        END IF;
    END IF;
END IF;
END;
/
```

INSERT

Vamos a realizar algunas comprobaciones para asegurarnos de que el trigger funciona correctamente.

Como se ve en la siguiente captura, aqui tenemos algunas líneas para insertar, comprobar, actualizar y borrar datos de la tabla:

```
/* Para comprobar las tablas */

INSERT INTO t_ciudadanos1@link_hugo
VALUES (3, 'Evelin', 'Dutuc', 'a', '21111111A');
COMMIT;

SELECT * FROM ciudadano;

SELECT * FROM t_ciudadanos1@link_hugo;
SELECT * FROM t_ciudadanos2@link_pablo;
SELECT * FROM t_ciudadanos3@link_martin;

UPDATE ciudadanos SET dni = '22345678a' WHERE id = 200;

DELETE ciudadanos WHERE ID = 200;
```

Empezaremos insertando datos.

```
/* Para comprobar las tablas */

INSERT INTO ciudadanos
VALUES (10, 'EvelinAndres', 'Dutuc', 'a', 'a87654321');
COMMIT;

SELECT * FROM ciudadanos;

SELECT * FROM t_ciudadanos1@link_hugo;
SELECT * FROM t_ciudadanos2@link_pablo;
SELECT * FROM t_ciudadanos3@link_martin;

UPDATE ciudadanos SET dni = '22345678a' WHERE id = 200;

DELETE ciudadanos WHERE ID = 200;
```

Salida de Script xResultado de la Consulta x

SQL | Todas las Filas Recuperadas: 2 en 0,01 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1	10 EvelinAndres	Dutuc	a	a87654321
2	199 ciudadano99	a	a	a12345678

Ahora consultamos la base de datos del primer vecino (En este caso todos los ID's entre 1 y 99). Podremos ver que efectivamente se ha copiado la insercción:

```
SELECT * FROM t_ciudadanos1@link_hugo;
SELECT * FROM t_ciudadanos2@link_pablo;
SELECT * FROM t_ciudadanos3@link_martin;

UPDATE ciudadanos SET dni = '22345678a' WHERE id = 200;

DELETE ciudadanos WHERE ID = 200;
```

Salida de Script xResultado de la Consulta x

SQL | Todas las Filas Recuperadas: 3 en 9,37 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1	1 Hugo	a	a	11111111A
2	3 Evelin	Dutuc	a	21111111A
3	10 EvelinAndres	Dutuc	a	a87654321

DELETE

Vamos a probar borrando la línea que hemos creado con anterioridad. La hemos borrado desde la tabla "ciudadanos", es decir, desde la tabla principal:

```
DELETE ciudadanos WHERE ID = 10;
```

Salida de Script xResultado de la Consulta x

Tarea terminada en 9,542 segundos

```
1 fila insertadas.  
  
Confirmación terminada.  
  
1 fila actualizadas.  
  
1 fila eliminado
```

Consultamos con un **SELECT** a nuestro **DBLINK** para comprobar que se han aplicado los cambios.

```
SELECT * FROM t_ciudadanos1@link_hugo;  
SELECT * FROM t_ciudadanos2@link_pablo;  
SELECT * FROM t_ciudadanos3@link_martin;  
  
UPDATE ciudadanos SET dni = '1245678x' WHERE id = 10;  
  
DELETE ciudadanos WHERE ID = 10;
```

Salida de Script xResultado de la Co... x

Todas las Filas Recuperadas: 2 en 9,355 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1	1 Hugo	a	a	11111111A
2	3 Evelin	Dutuc	a	21111111A

Comprobación desde otros usuarios vecinos

Con el usuario pablo (100 - 199)

En primer lugar insertamos la fila con el `id = 150`, aunque el `id 100` y `199` también se ha seguido el mismo proceso.

```
SELECT * FROM ciudadanos;
```

```
SELECT * FROM t_ciudadanos1@link_hugo;
```

```
SELECT * FROM t_ciudadanos2@link_pablo;
```

```
SELECT * FROM t_ciudadanos3@link_martin;
```



```
UPDATE ciudadanos SET dni = '1245678x' WHERE id = 10;
```



```
DELETE ciudadanos WHERE ID = 10;
```

Salida de Script xResultado de la Consulta x

SQL | Todas las Filas Recuperadas: 3 en 0,029 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1 100	EvelinAndres	Dutuc	a	a87654321
2 150	EvelinAndres150	Dutuc	a	x87654321
3 199	ciudadano99	a	a	a12345678

Si consultamos la tabla vecina:

```
SELECT * FROM t_ciudadanos1@link_hugo;
```

```
SELECT * FROM t_ciudadanos2@link_pablo;
```

```
SELECT * FROM t_ciudadanos3@link_martin;
```



```
UPDATE ciudadanos SET dni = '1245678x' WHERE id = 10;
```



```
DELETE ciudadanos WHERE ID = 10;
```

Salida de Script xResultado de la Consulta x

SQL | Todas las Filas Recuperadas: 4 en 0,031 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1 100	EvelinAndres	Dutuc	a	a87654321
2 150	EvelinAndres150	Dutuc	a	x87654321
3 1	Pablo	Luna	Luna	11111111A
4 199	ciudadano99	a	a	a12345678

Con el usuario martin (200 o más)

Insertamos con `id = 250`:

```
SELECT * FROM ciudadanos;

SELECT * FROM t_ciudadanos1@link_hugo;
SELECT * FROM t_ciudadanos2@link_pablo;
SELECT * FROM t_ciudadanos3@link_martin;

UPDATE ciudadanos SET dni = '1245678x' WHERE id = 10;

DELETE ciudadanos WHERE ID = 150;
```

Salida de Script x Resultado de la Consulta x

SQL | Todas las Filas Recuperadas: 2 en 0,014 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1	199ciudadano99	a	a	a12345678
2	250EvelinAndres250	Dutuc	a	x87654321

Y comprobamos:

```
SELECT * FROM t_ciudadanos1@link_hugo;
SELECT * FROM t_ciudadanos2@link_pablo;
SELECT * FROM t_ciudadanos3@link_martin;

UPDATE ciudadanos SET dni = '1245678x' WHERE id = 10;

DELETE ciudadanos WHERE ID = 150;
```

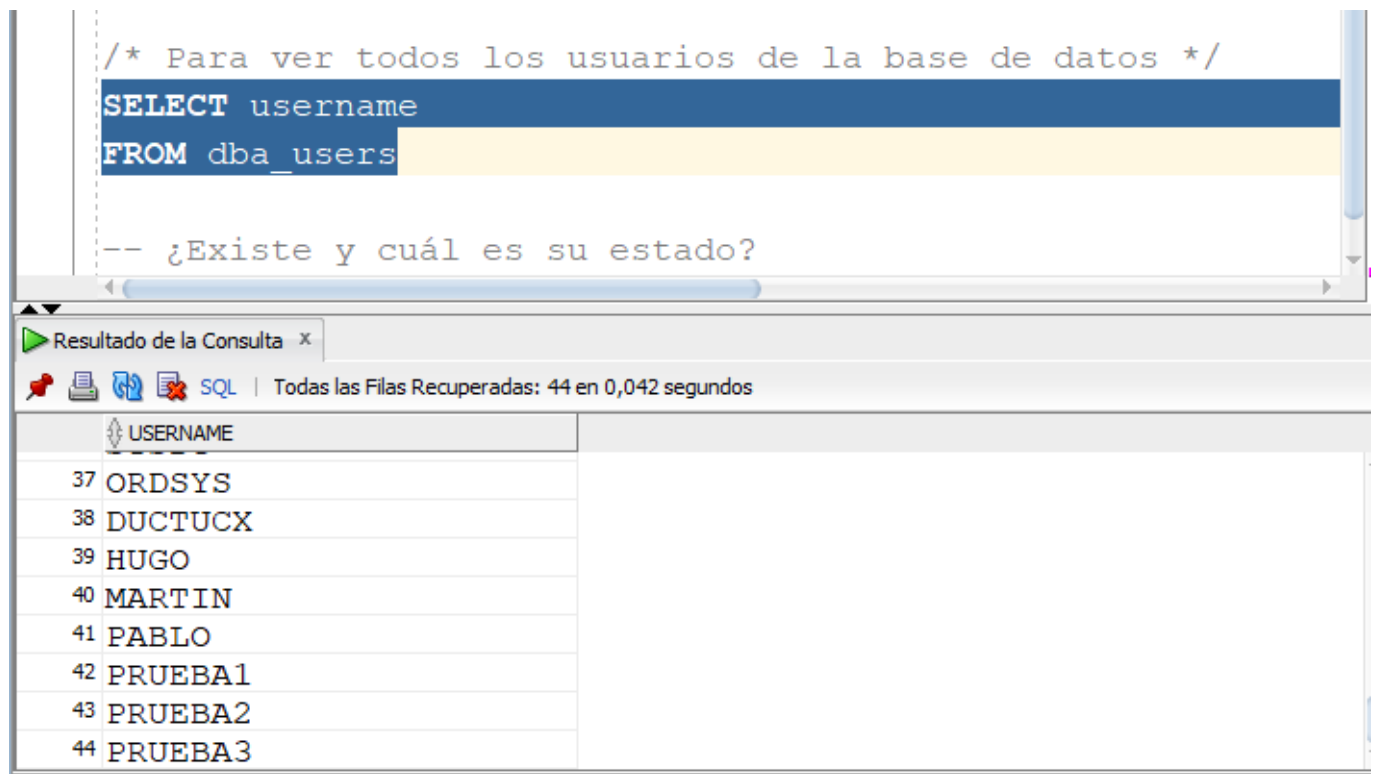
Salida de Script x Resultado de la Consulta x

SQL | Todas las Filas Recuperadas: 2 en 0,03 segundos

ID	NOMBRE	APELLIDO1	APELLIDO2	DNI
1	250EvelinAndres250	Dutuc	a	x87654321
2	1Martin	a	a	11111111A

BD con los usuarios

Aquí están todos los usuarios creados:



The screenshot shows a SQL query window with the following text:

```
/* Para ver todos los usuarios de la base de datos */  
SELECT username  
FROM dba_users  
  
-- ¿Existe y cuál es su estado?
```

Below the query window, the results are displayed in a table titled "Resultado de la Consulta". The table has one column, "USERNAME", and 44 rows. The first few rows are visible:

	USERNAME
37	ORDSYS
38	DUCTUCX
39	HUGO
40	MARTIN
41	PABLO
42	PRUEBA1
43	PRUEBA2
44	PRUEBA3

Distribución de usuarios

Usuario local

- Ductucx

Usuarios de los vecinos

- hugo
- pablo
- martin